Software Requirements Specification

Versatile and Fast Algorithm for Optimal Ship Routing Indian Ocean Region

Document Type	**Software Requirements Specification (SRS)**
Project Title	**Optimal Ship Routing Algorithm for the Indian Ocean**
Algorithm	**Multi-Objective Particle Swarm Optimization (MOPSO)**
Language	**Python 3.9+ | Open Source**
Version	**1.0 (Initial Release)**
Institute	**B.Tech Project — Computer Science Engineering (AI-IBM)**
Status	**In Development**

Table of Contents

1. Introduction 1

1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience 1
1.4 Project Background 1
1.5 References 2

2. Overall Description 3

2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 4
2.4 Operating Environment 4
2.5 Design / Implementation Constraints 4
2.6 Assumptions and Dependencies 5

3. External Interface Requirements 6

3.1 User Interfaces 6
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communication Protocols and Interfaces 7

4. System Features 8

4.1 MOPSO Optimization Engine 8
4.2 Weather Data Integration Module 9
4.3 Constraint Handling Module 10
4.4 Objective Functions 10
4.5 Output and Visualization 11

5. Non-Functional Requirements…….…… 12

5.1 Performance Requirements……...…….. 12
5.2 Safety Requirements…….. 12
5.3 Software Quality Attributes…….……. 12
5.4 Project Documentation…….……. 13

6. Appendix A — Glossary…..……...……... 14

7. Appendix B — Algorithm Parameters…..…….. 16

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the Optimal Ship Routing System built for the Indian Ocean region. The system computes the best possible voyage route between any two ports by simultaneously minimizing travel time, weather-related safety risk, and fuel consumption using a Multi-Objective Particle Swarm Optimization (MOPSO) algorithm implemented in Python. This document serves as the technical reference for development, testing, and academic evaluation of the project.

1.2 Document Conventions

This document follows IEEE 830 SRS conventions. All requirements are stated in present tense. The word 'shall' denotes a mandatory requirement. The word 'should' denotes a recommended but optional requirement. Bold text marks key technical terms on first use. Geographic coordinates are in decimal degrees using the WGS84 datum. Module names are written in monospace.

1.3 Intended Audience

This document is written for the following readers:

- Project Group Members — to understand what each module does and how to contribute.
- Project Supervisor / Evaluator — to verify that the system meets academic and engineering standards.
- Future Developers — to extend the system with new objectives, ship types, or a web interface.
- Shipping Industry Stakeholders — to understand the system's capabilities for real-world deployment in the Indian Ocean.

1.4 Project Background

Over 80% of global trade volume is carried by sea, with the Indian Ocean being one of the world's busiest shipping lanes — connecting India, the Arabian Gulf, East Africa, and Southeast Asia. Ships operating in this region face seasonal monsoon weather, Arabian Sea cyclones, and the Agulhas Current in the south, all of which significantly affect fuel consumption, voyage time, and safety.

Indian shipping companies spend a large portion of their operating budgets on fuel. Even a 5% reduction in fuel consumption across a fleet translates to crores of rupees in annual savings. However, the cheapest route is not always the safest. This project develops an algorithm that

can balance multiple objectives simultaneously — rather than optimizing just one — and return a set of trade-off routes for the operator to choose from.

No publicly available, customizable routing application exists for the Indian Ocean region. This system fills that gap using open-source Python tools, making it accessible to Indian maritime institutions and shipping companies.

1.5 References

1)	IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications.
2)	Kennedy J. & Eberhart R. (1995) — Particle Swarm Optimization. IEEE ICNN Proceedings.
3)	Coello C.A. et al. (2004) — Handling Multiple Objectives with PSO. IEEE Trans. on Evolutionary Computation.
4)	IMO MSC-MEPC.2/Circ.13 — Guidelines for Voyage Planning, International Maritime Organization.
5)	ECMWF ERA5 Reanalysis Dataset — Copernicus Climate Data Store (cds.climate.copernicus.eu).
6)	CMEMS Global Ocean Physics Analysis — Copernicus Marine Service (marine.copernicus.eu).
7)	GEBCO 2023 Global Bathymetric Chart — General Bathymetric Chart of the Oceans (gebco.net).

2. Overall Description

2.1 Product Perspective

The Optimal Ship Routing System is a standalone Python application. It accepts voyage inputs — origin port, destination port, ship specifications, and weather data — and outputs a Pareto-optimal set of routes, each representing a different trade-off between travel time, safety, and fuel. The system is designed as a backend optimization engine that can be wrapped in a web API (FastAPI) or integrated into a ship's Electronic Chart Display and Information System (ECDIS) in future versions.

The geographic scope covers the entire Indian Ocean: latitude 40°S to 30°N, longitude 30°E to 120°E. This encompasses all major Indian Ocean shipping lanes between Indian ports (Mumbai, Chennai, Kolkata), the Arabian Gulf, East African ports (Mombasa, Dar es Salaam), and the Strait of Malacca.

2.2 Product Functions

Function	Description
Swarm Initialization	Generate N candidate routes as randomly perturbed straight-line paths between the origin and destination points.
Objective Evaluation	Compute travel time (in hours), safety penalty (dimensionless), and fuel consumption (in tonnes) for each candidate route.
Pareto Archive Management	Identify non-dominated routes and maintain a bounded archive using crowding-distance pruning to preserve diversity.
Particle Velocity Update	Move each candidate route toward better positions using the PSO update equation with adaptive parameters.
Constraint Enforcement	Detect and repair waypoints that fall on land or shallow water after every position update.
Route Ranking	Score all Pareto-optimal routes using a configurable weighted preference model and rank them accordingly.
Visualization & Export	Plot Pareto routes on an Indian Ocean map, generate scatter plots, and export results in GeoJSON format for GIS tools.

2.3 User Classes and Characteristics

User Class	Role	Interaction Level
Ship Captain / Navigator	Selects a route from the Pareto front based on voyage priorities (safety and speed).	Basic user interface interaction; does not modify system logic.
Fleet Manager	Compares fuel cost and performance across Pareto routes to optimize fleet operations.	Moderate interaction; analyzes outputs and reports.
Developer / Researcher	Extends the system with new objectives, ship models, or environmental datasets.	High interaction; modifies codebase and algorithms directly.
Project Evaluator	Reviews the SRS, system design, and algorithm correctness for academic evaluation.	Limited interaction; focuses on documentation and results validation.

2.4 Operating Environment

- Operating System: Linux, Windows 10/11, or macOS (any OS with Python 3.9+).
- Language runtime: Python 3.9 or higher.
- Minimum RAM: 4 GB. Recommended: 8 GB for large swarm sizes (n_particles > 150).
- CPU: Any modern multi-core processor. Parallel evaluation uses all available cores.
- Storage: 200 MB for application code. Additional 2–5 GB if downloading real ERA5 weather netCDF files.
- Internet: Not required in demo mode. Required for Copernicus ERA5 / CMEMS data download in production mode.
- No commercial software or paid libraries are needed — the entire stack is open source.

2.5 Design and Implementation Constraints

- The entire system must be implemented in Python using only open-source libraries (NumPy, SciPy, Matplotlib, netCDF4, GeoPandas).
- The optimizer must complete a standard run (80 particles, 200 iterations, 8 waypoints) within 5 minutes on a 4-core machine.
- The system must handle at least 3 simultaneous optimization objectives without requiring changes to the core MOPSO engine.
- All route waypoints must remain within the defined Indian Ocean bounding box at all times.
- The land avoidance constraint must be enforced after every particle position update — no output route may pass through land.
- The Pareto archive size is bounded at 200 solutions to control memory usage and computation time.

- The system must work in demo mode without any internet connection, using the built-in synthetic weather grid.

2.6 Assumptions and Dependencies

- Ship parameters (service speed, beam, draft, fuel rate, safety limits) are provided accurately by the operator before each run.
- Weather data is available either as the built-in synthetic grid (demo) or as a downloaded ERA5/CMEMS netCDF4 file (production).
- Port entry and harbour navigation are outside the scope of this system — only open-ocean routing between port coordinates is handled.
- Tidal effects, vessel traffic separation schemes, and port congestion are not modelled in Version 1.0.
- The system assumes the ship travels at its service speed, adjusted by weather conditions and ocean currents.
- A ship's drift characteristics and windage are approximated through the Beaufort speed-reduction table.

3. External Interface Requirements

3.1 User Interface

Version 1.0 provides a command-line interface (CLI). The user edits parameters directly inside main.py and runs the script. All output is printed to the terminal and saved to files in the working directory. No graphical UI is required for this version.

The following output files are produced automatically on each run:

Output File	Format	Description
routes_map.png	PNG image	All Pareto routes plotted on Indian Ocean basemap, colour-coded by travel time.
pareto_front.png	PNG image	Two scatter plots: time vs safety, and time vs fuel, coloured by third objective.
optimal_routes.geojson	GeoJSON	All Pareto routes in standard geographic format — load in QGIS or Google Earth.
route_report.txt	Plain text	Top 5 recommended routes with travel time, safety score, and fuel consumption.

3.2 Hardware Interfaces

The system runs on standard desktop or laptop hardware — no specialized maritime hardware is required for Version 1.0. For onboard ship deployment (Version 2.0), the system would run on a navigation computer connected to the ship's ECDIS via Ethernet. GPS position input via NMEA 0183 protocol would enable automatic re-optimization triggered by the ship's actual position every 6 hours.

3.3 Software Interfaces

Library	Version	Role in This Project
Python	3.9+	Core programming language for all modules.
NumPy	>=1.24	All numerical arrays, vector math, coordinate arithmetic.
SciPy	>=1.10	Scientific utilities used in interpolation and distance calculations.
Matplotlib	>=3.7	Generates route map PNG and Pareto front scatter plots.

3.4 Communication Interfaces

Version 1.0 operates entirely offline. No network communication is needed in demo mode. For production weather data, the system uses the CDS API (Copernicus Climate Data Store) over HTTPS to download ERA5 reanalysis netCDF files. Output GeoJSON files are compatible with any standard GIS platform and can be loaded directly into maritime charting software. In Version 2.0, a FastAPI REST endpoint will accept voyage requests over HTTPS and return route data in GeoJSON format.

4. System Features

4.1 MOPSO Optimization Engine

The MOPSO engine maintains a swarm of N particles, where each particle is a candidate ship route encoded as a sequence of intermediate waypoints between origin and destination. At every iteration, each particle is evaluated on all registered objective functions. Non-dominated solutions are stored in a bounded Pareto archive. Particles update their positions using the standard PSO velocity equation:
v(t+1) = w * v(t) + c1 * r1 * (pBest - x(t)) + c2 * r2 * (gBest - x(t))
where w is the inertia weight (decreases linearly from 0.6 to 0.3 to encourage exploration early and exploitation late), c1 is the cognitive acceleration toward the particle's own personal best, and c2 is the social acceleration toward the global best selected from the Pareto archive using crowding-distance weighted tournament selection. This selection strategy promotes archive diversity by favouring solutions in less-crowded objective space regions.

Key classes and methods in this module:

Class / Method	Purpose
MOPSORouter	Main optimizer class. Configure via n_particles, n_waypoints, n_iter, w, c1, c2, bounds.
MOPSORouter.optimise()	Runs the full optimization loop. Returns a list of WaypointRoute objects (Pareto archive).
MOPSORouter.add_objective()	Registers a new objective function f(route, ship) -> float to be minimized.
WaypointRoute	Data class holding a route's waypoints array (N x 2) and objectives vector.
update_archive()	Adds a candidate solution to the Pareto archive, removing any dominated solutions.
crowding_distance()	Computes crowding distance for all archive solutions to measure objective-space density.
select_gbest()	Selects global best guide from archive using crowding-distance weighted random selection.

Functional Requirements:

- FR-1.1: The system shall support a configurable swarm size (default: 80 particles, valid range: 20–500).
- FR-1.2: The number of intermediate waypoints per route shall be configurable (default: 8).

- FR-1.3: The optimizer shall run for a configurable number of iterations (default: 200).
- FR-1.4: The Pareto archive shall store only non-dominated solutions at all times.
- FR-1.5: The archive shall be pruned to a maximum of 200 solutions using crowding distance when full.
- FR-1.6: The inertia weight shall decrease linearly from 0.6 to 0.3 across all iterations.
- FR-1.7: New objective functions shall be pluggable without modifying the MOPSO core engine.
- FR-1.8: The optimizer shall return a non-empty Pareto archive for any valid Indian Ocean voyage.

4.2 Weather Data Integration Module

File: src/objectives.py — WeatherGrid class | Priority: HIGH

The WeatherGrid class stores a 2-D grid of meteorological and oceanographic fields over the Indian Ocean and provides bilinear spatial interpolation at any (lat, lon) point. In demo mode it generates a realistic synthetic grid; in production it reads real data from a netCDF4 file via WeatherGrid.from_netcdf()

Weather fields modelled:

Field	**Unit**	**Source (Production)**	**Effect on Routing**
Significant Wave Height (Hs)	**metres**	**ERA5 variable: swh**	**Reduces ship speed; increases added resistance and safety penalties in rough sea conditions.**
Wind Speed (U10)	**m/s**	**ERA5 variables: u10, v10**	**Increases hull resistance; reduces vessel speed; may trigger safety penalties at high wind speeds.**
Wind Direction	**degrees**	**ERA5 variables: u10, v10**	**Used to compute head-sea versus following-sea conditions affecting ship performance.**
Surface Current Speed	**m/s**	**CMEMS: uo, vo at surface**	**Adds to or subtracts from ship speed depending on alignment with the vessel's heading.**
Surface Current Direction	**degrees**	**CMEMS: uo, vo at surface**	**Determines the current component along the ship's heading and influences effective speed.**

- FR-2.1: The weather model shall provide all five fields at any (lat, lon) within the Indian Ocean domain.

- FR-2.2: Bilinear interpolation shall be used to compute field values between grid points.
- FR-2.3: The system shall support loading real ERA5/CMEMS data from netCDF4 files without code changes.
- FR-2.4: Interpolation shall execute in under 1 millisecond per query point.

4.3 Constraint Handling Module

File: src/constraints.py | Priority: HIGH — Safety critical.

After each PSO position update, the constraint function checks every waypoint against a set of polygon-based land masks covering the Indian Ocean region. Any waypoint found on land is repaired using a random perturbation search followed by a fallback toward the Indian Ocean centre. The module uses a ray-casting point-in-polygon algorithm.

Land polygons covered (Version 1.0):

- Indian Subcontinent (crude approximation — 8 vertices)
- Sri Lanka • Arabian Peninsula (western boundary)
- East Africa coast strip
- Malay Peninsula and Strait of Malacca
- Sumatra (western coast)
- Madagascar (eastern coast)

- FR-3.1: No route waypoint shall be placed on a land mass in any output route.
- FR-3.2: Constraint repair shall try up to 20 random perturbations before falling back to ocean centre.
- FR-3.3: A minimum ocean depth constraint equal to ship draft + 5m safety margin shall be enforced.
- FR-3.4: The constraint function interface shall allow addition of new exclusion zones (EEZ, storm areas) without modifying the optimizer.

4.4 Objective Functions

File: src/objectives.py | Priority: HIGH

Three objective functions are implemented, each returning a scalar value to be minimized. They are evaluated by sampling each route segment at 20 equally-spaced interior points to capture weather variability along the segment.

Objective	Function Name	Model Used	Output Unit
Travel Time	make_travel_time_objective()	Service speed reduced by Beaufort-scale factors from wave height; adjusted for wind and surface currents.	Hours
Safety Penalty	make_safety_objective()	Quadratic penalty based on wave height and wind speed exceeding safety thresholds.	Dimensionless
Fuel Consumption	make_fuel_objective()	Cubic speed–power relationship for fuel rate, multiplied by time and adjusted with added resistance due to waves and wind.	Tonnes

- FR-4.1: All three objective functions shall be evaluated for every particle at every iteration.
- FR-4.2: Each objective shall sample every route segment at 20 equally-spaced points.
- FR-4.3: The travel time objective shall account for ocean current assistance or resistance along the ship's heading.
- FR-4.4: The safety penalty shall be zero only when all sampled points are below the ship's wave height and wind speed limits.
- FR-4.5: The fuel objective shall scale with the cube of effective vessel speed.

4.5 Output and Visualization

File: src/visualise.py | Priority: MEDIUM

- FR-5.1: A PNG route map shall be produced showing all Pareto routes on an Indian Ocean basemap, colour-coded by travel time.
- FR-5.2: The recommended route shall be distinctly highlighted on the map in dashed white.
- FR-5.3: Two Pareto scatter plots shall be produced: travel time vs safety penalty, and travel time vs fuel consumption.
- FR-5.4: All routes shall be exported as a GeoJSON FeatureCollection with objective values stored as feature properties.
- FR-5.5: A plain-text route_report.txt shall be generated listing the top 5 routes with full objective values.
- FR-5.6: The preference ranking shall use a configurable weighted sum: default weights 0.4 (time), 0.4 (safety), 0.2 (fuel).

5. Non-Functional Requirements

5.1 Performance Requirements

Performance is critical because ship routing must be re-computed periodically as weather evolves. The following benchmarks are defined for a standard voyage (Mumbai to Mombasa) on a 4-core 2.5 GHz machine with 8 GB RAM.

Metric	Requirement
Full optimization run (80 particles, 200 iterations, 8 waypoints)	Must complete within 5 minutes.
Weather grid interpolation	Under 1 ms per query point.
Constraint repair per waypoint	Under 50 ms per waypoint.
GeoJSON export	Under 2 seconds for any archive size up to 200 routes.
Speedup with multiprocessing	Near-linear with number of CPU cores when parallel evaluation is enabled.

5.2 Safety Requirements

- The system shall never return a route that passes through a land mass.
- Routes where any sampled point has wave height > ship.max_wave_height_m shall receive a non-zero safety penalty and shall not appear in the top recommendation unless no safer alternative exists.
- Routes with wind speed > ship.max_wind_speed_ms at any sampled point are considered operationally dangerous and shall be assigned maximum safety penalty.
- The system shall not recommend routes through known active cyclone paths if cyclone position data is provided.
- All safety thresholds are configurable per vessel type via the ShipParams data class.

5.3 Software Quality Attributes

Attribute	Requirement
Reliability	The optimizer shall always produce a non-empty Pareto front for any valid Indian Ocean origin-destination pair.

Extensibility	Adding a new objective (e.g., CO2 emissions, passenger comfort) requires only writing a new function and calling add_objective() — no changes to MOPSO core.
Portability	The system shall run without modification on Linux, Windows, and macOS.
Maintainability	Each module has a single responsibility. Cyclomatic complexity per function shall not exceed 15.
Testability	All objective functions and constraint handlers shall have unit tests achieving at least 80% line coverage.
Readability	All functions shall include docstrings. Variable names shall be descriptive and self-documenting.

5.4 Project Documentation

- All Python modules include docstrings describing purpose, parameters, and return values.
- A README.md on the GitHub repository provides setup instructions, module descriptions, and sample results.
- This SRS document is version-controlled alongside the source code.
- A route_report.txt is automatically generated on every optimization run. SRS Document v1.0
- Inline comments explain non-obvious algorithmic steps (Pareto dominance, crowding distance, Beaufort table).

6. Appendix A — Glossary

The following terms are used throughout this document and the project codebase.

Term	Definition
MOPSO	Multi-Objective Particle Swarm Optimization — a metaheuristic search algorithm that evolves a swarm of candidate solutions toward the Pareto front.
Particle	A single candidate route in the swarm, encoded as an array of intermediate waypoints (latitude and longitude pairs).
Pareto Front	The set of all non-dominated solutions — routes where improving one objective necessarily worsens at least one other.
Pareto Dominance	Solution A dominates solution B if A is no worse in all objectives and strictly better in at least one objective.
Crowding Distance	A measure of how isolated a solution is in objective space, used to preserve diversity in the Pareto archive.
Inertia Weight (w)	A PSO parameter controlling how much a particle retains its previous velocity; decreases over time to shift from exploration to exploitation.
Personal Best (pBest)	The best position a single particle has visited so far, evaluated using Pareto dominance.
Global Best (gBest)	The best solution selected from the Pareto archive, used as a global guide for all particles.
Waypoint	An intermediate geographic coordinate (latitude, longitude) that defines the shape of a candidate route.
Significant Wave Height (Hs)	The average height of the highest one-third of waves at a location, measured in metres; a standard indicator of sea state.
Beaufort Scale	A 0–12 wind force scale; higher values indicate rougher seas and reduced ship speed.
ERA5	ECMWF's fifth-generation atmospheric reanalysis dataset used for global weather data (historical and forecast).
CMEMS	Copernicus Marine Environment Monitoring Service — provides ocean current, wave, and marine data.
GeoJSON	An open standard format for encoding geographic features and their properties using JSON; widely used in GIS tools.

ECDIS	Electronic Chart Display and Information System — a digital navigation system used on modern ships.
EEZ	Exclusive Economic Zone — a sea zone extending up to 200 nautical miles from a country's coastline, where it has special rights.
NMEA 0183	A standard communication protocol used by marine navigation devices (GPS, sonar, etc.).
WGS84	World Geodetic System 1984 — the standard coordinate system used for latitude and longitude globally.
netCDF4	Network Common Data Form version 4 — a standard scientific file format for storing gridded meteorological and oceanographic data.

7. Appendix B — Algorithm Parameters Reference

The table below lists all configurable parameters of the MOPSO optimizer along with their default values, valid ranges, and guidance on tuning. These are set in main.py when creating the MOPSORouter instance.

Parameter	Default	Valid Range	Effect / Tuning Guidance
n_particles	80	20 – 500	Larger swarms improve exploration but increase computation time. Use 80–120 for balanced performance in production.
n_waypoints	8	4 – 20	More waypoints create smoother routes but increase search complexity. Typically, 6–10 gives optimal results.
n_iter	200	50 – 1000	More iterations improve convergence quality. 200 is sufficient for most cases; use 500 for critical route planning.
w (inertia)	0.6 → 0.3	0.3 – 0.9	Higher values encourage exploration, while lower values focus on exploitation. A linear decay strategy is recommended.
c1 (cognitive)	1.5	0.5 – 2.5	Controls attraction toward personal best (pBest). Higher values make particles behave more independently.
c2 (social)	2.0	0.5 – 2.5	Controls attraction toward global best (gBest). Slightly higher than c1 improves convergence stability.
archive_max	200	50 – 500	Larger archive size maintains better diversity of solutions but increases memory and computation cost.
lat_bounds	(-40, 30)	Fixed	Represents Indian Ocean latitude limits. Modify only if extending the model to other regions.
lon_bounds	(30, 120)	Fixed	Represents Indian Ocean longitude limits. Modify only if extending the model to other regions.
n_samples	20	10 – 50	Number of sampling points per route segment for objective evaluation. Higher values improve accuracy but increase computation time.

ShipParams — Configurable Ship Characteristics:

Parameter	Example Value	Description
name	Indian bulk carrier	Vessel name or type label.
max_speed_kn	15.0 knots	Maximum possible vessel speed in knots.
service_speed_kn	12.0 knots	Normal cruise speed used for time and fuel calculations.
fuel_rate_t_per_day	28.0 tonnes/day	Fuel consumption at service speed in calm water.
beam_m	32.0 m	Ship width in metres — affects added resistance in waves.
draft_m	10.5 m	Depth below waterline — used for minimum depth constraint.
max_wave_height_m	5.0 m	Significant wave height above which safety penalty is applied.
max_wind_speed_ms	18.0 m/s	Wind speed above which safety penalty is applied.